NoSQL Database — BCSE406L

Review 2: Design and Implementation

Real-Time Collaborative Whiteboard

An append-only document model in MongoDB with Redis for ordering, presence, and cross-instance event fanout

S.No	Register No	Student Name
1	23BCE0801	Devesh Atul Mahajan
2	23BCE0826	Prakrit Bahl
3	23BCE0811	Lakshya Diwani

Item	Detail
Batch / Team	NS02
Branch / Semester	B.Tech CSE / 7th
Academic Year	2026-2027
Faculty Guide	Dr. D. Vivek
Primary NoSQL databases	MongoDB (document) and Redis (key-value)
Collections implemented	rooms, strokes, snapshots
Documents in the database	80,019 stroke documents
Indexes implemented	7 purpose-built, plus default _id
Aggregation pipelines	9 implemented and demonstrated
Measurements taken	21 September 2026, 16:58 UTC

Contents

1. Introduction and Scope
2. System Design
3. Database Design and JSON Collections
4. CRUD Implementation
5. Indexing and Query Plans
6. Aggregation Queries
7. Performance and Scalability
8. Prototype Walkthrough
9. Progress Report
10. Source Code Listings
11. References

1. Introduction and Scope

1.1 What the system does

The Real-Time Collaborative Whiteboard is a web application in which several people draw at the same time on one shared canvas, every change appearing on every other connected screen without a refresh. The decision that shapes everything else is that the canvas is never stored as a picture. It is an ordered, append-only stream of stroke documents, each tagged with its room and the sequence number it was assigned; what a user sees is derived state, reconstructed by replaying that stream.

That choice is what makes this a NoSQL project rather than a drawing exercise. A pen stroke, an eraser stroke, a rectangle and a text label carry genuinely different fields and share one collection with no migration between them. Undo becomes a flag rather than a deletion, so history survives. Ordering has to come from somewhere other than a clock. None of these follow naturally from a fixed-schema table, and each is examined below.

1.2 Objectives carried forward from Review 1

Objective stated in Review 1	Status at Review 2
Real-time multi-user drawing across several clients	Complete; verified with two browsers on two backend processes.
Model state as a stream of stroke documents, reconstructable by replay	Complete; replay is the only way the canvas is built (sections 3, 7).
Low-latency sync of drawing and cursors over WebSockets with Redis Pub/Sub	Complete; one delivery path through Redis (section 2.7).
Undo and redo by soft delete rather than physical deletion	Complete and extended: scoped per user, claimed atomically (section 4.6).
Evaluate performance through indexing, query optimisation, optional snapshotting	Complete; seven indexes with measured plans (section 5), snapshotting promoted from optional to implemented (section 7.2).
Justify MongoDB and Redis over a relational database	Sections 2.3, 3.5 and 6.6.

1.3 What Review 2 adds

Two of the Review 2 requirements changed the design rather than merely adding code.

- **An append-only log has no natural update or delete.** Event sourcing only inserts, and undo is a flag. Operations that genuinely modify and genuinely remove documents were therefore added: renaming a board, restyling and moving a stroke, compaction that turns soft deletes into real ones, and a cascading room delete. Three deletion semantics can now be contrasted.
- **Nothing in the original design needed aggregation,** since replaying a canvas is a find with a sort. An analytics panel was designed into the application, backed by 9 pipelines answering questions the application actually has about its own data rather than pipelines written to satisfy a checklist (section 6).

A third change came from measurement. Snapshotting was optional in Review 1; measuring the largest room showed a full replay transfers 66 MB, so it was implemented and is now part of the normal join path (section 7.2).

2. System Design

2.1 Runtime architecture

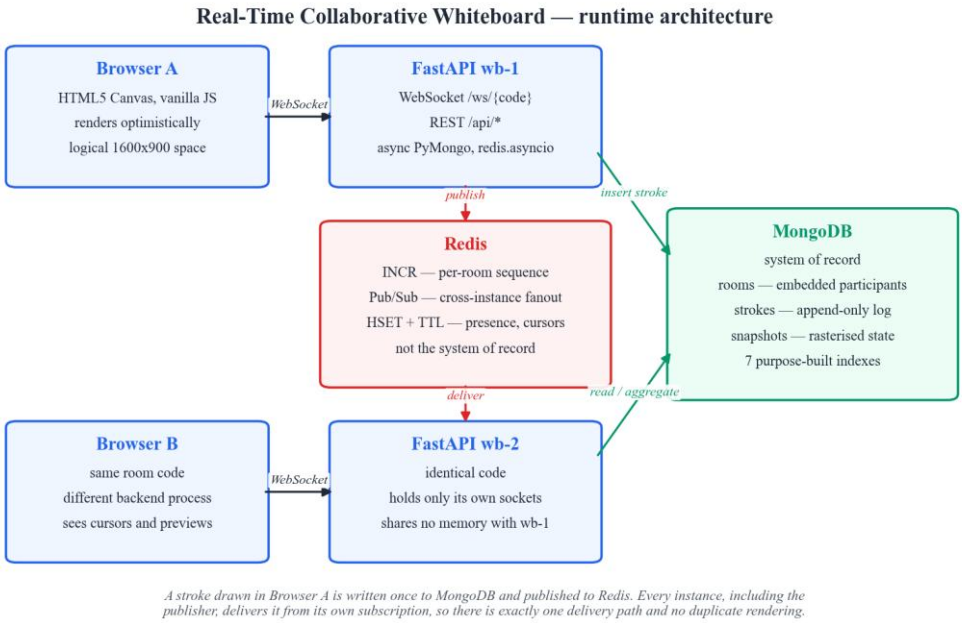


Figure 2.1 — Runtime architecture. Redis is drawn between the two application instances because that is its role: the bus they share.

The deployment shown is the one used for the demonstration: two independent FastAPI processes, each holding its own WebSocket connections and sharing no memory with the other, plus one MongoDB and one Redis. Two processes rather than one is deliberate. A single process could broadcast from an in-memory dictionary and would never exercise the broadcast layer at all, leaving the claim that the design scales across instances untested.

2.2 Components and responsibilities

Layer	Technology	Responsibility
Client	HTML5 Canvas, ES modules, no build step	Captures pointer input, renders optimistically, rasterises snapshots, and displays the analytics panel.

Transport	WebSocket, one connection per client	Carries ephemeral previews and cursors as well as durable stroke commits.
Application	FastAPI on Uvicorn, fully async	Validates input, assigns sequence numbers, persists strokes, and publishes events.
System of record	MongoDB 7 (replica set)	Stores rooms, the stroke log, and snapshots. The only durable state.
Coordination	Redis 7	Atomic per-room counters, Pub/Sub fanout between instances, and presence with a TTL.

Both database drivers are asynchronous — the PyMongo async client and redis.asyncio — which matters more than it appears. A blocking driver call inside an async WebSocket handler stalls the event loop for every other user on that process, destroying precisely the concurrency the project sets out to demonstrate.

2.3 Why two data stores

Two databases need justifying, since the obvious criticism is one more thing to operate. The division is by durability requirement, not convenience.

- **MongoDB holds anything that must survive a restart:** rooms, the stroke log, snapshots. It is the only source of truth, and the whiteboard can be rebuilt from it alone.
- **Redis holds only data that is worthless once a connection drops,** plus the one thing that must be atomic across processes. Cursors and presence expire by TTL and are never persisted, because writing a cursor position that is stale within 16 ms is write amplification for no benefit. The sequence counter is in Redis because INCR is atomic across every instance.

The practical test is what breaks when each store is lost. If Redis restarts, cursors disappear, presence rebuilds as clients heartbeat, the counter is realigned from the highest persisted sequence, and no drawing is lost. If MongoDB is lost, the boards are gone. That asymmetry is the design working as intended.

2.4 Event ordering

Replay only reconstructs the right picture if strokes are replayed in the order they were made, and this is the subtlest part of the design. Two candidates were rejected: client timestamps cannot be trusted because participants' clocks disagree, sometimes by seconds, and a MongoDB ObjectId embeds a timestamp with only one-second granularity, so two strokes drawn 40 ms apart on different machines may be ordered arbitrarily — an intermittent failure that is easy to miss in testing.

Ordering therefore comes from a Redis counter per room, incremented atomically before each stroke is written. Because INCR is atomic and Redis is shared, every instance draws from the same sequence and the order is total and reproducible; this is the second and stronger reason Redis is in the architecture. Since Redis is a cache rather than the record, the counter is realigned upward from the highest persisted sequence on every join, so a Redis restart cannot cause a collision.

```
async def next_seq(room_id) -> int:
    return int(await redis().incr(seq_key(room_id)))


async def sync_seq_floor(room_id, value: int) -> None:
    """Keep the Redis counter ahead of what is already stored.
```

```
    Redis is a cache here, not the system of record. If it restarts the counter
    is gone, so on room load we push it back up to the highest persisted seq.
    """
    current = await redis().get(seq_key(room_id))
    if current is None or int(current) < value:
        await redis().set(seq_key(room_id), value)


# ------------------------------------------------------------------ CREATE
```

2.5 The write path

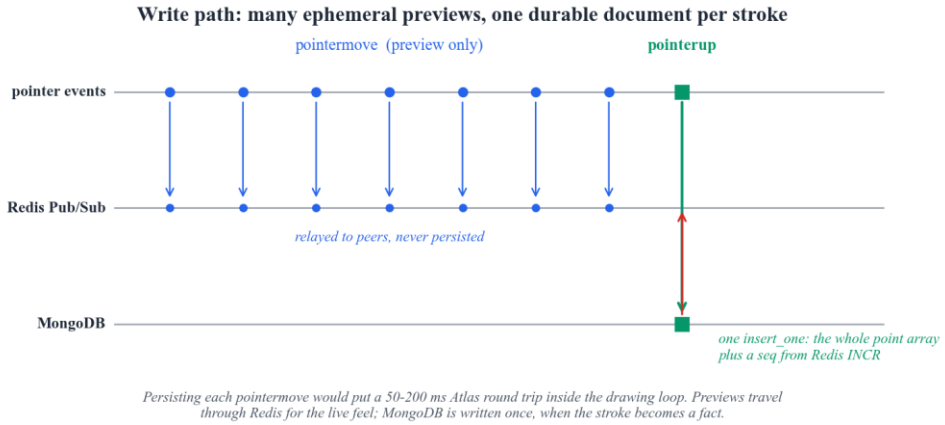


Figure 2.2 — One stroke produces many ephemeral preview messages but exactly one durable document.

While a pointer is down the client emits a preview roughly every frame. Previews are relayed through Redis and never written to MongoDB, because a stroke in progress is not yet a fact about the document. When the pointer lifts, the client commits once and the server writes a single document holding the whole point array. One document per pointer sample was rejected on latency grounds: the project targets Atlas, where an insert is a round trip of roughly 50 to 200 ms on the free tier, and putting that inside the drawing loop would make the pen feel broken. It would also multiply the document count by the average points per stroke, measured at about 37. Storing the array inline keeps the collection two orders of magnitude smaller and the pen responsive.

2.6 Concurrency: why there is no operational transformation

The Review 1 survey cited operational transformation and CRDT papers, so it is fair to ask where that machinery is. This workload does not need it, and the reason is precise. Operational transformation reconciles concurrent edits to shared mutable state, where an operation's meaning depends on operations elsewhere: inserting a character at index 5 means something different once somebody has inserted at index 2, so the operation must be transformed before it is applied. Appending an independent stroke has no such dependency. Stroke A then B gives the same canvas as B then A, because each carries its own geometry and paints its own pixels. The operations commute, so all that is required is a total order for replay, which the sequence counter provides. One genuine conflict does arise — two users pressing undo at the same instant could both try to retract the same stroke — and it is resolved with an atomic claim rather than transformation (section 4.6). Operational transformation would become necessary if the board gained shared editable text or draggable objects, which section 9.4 records as future work rather than claiming now.

2.7 One delivery path, and the duplicate-stroke trap

The natural first implementation of broadcast is for a handler to send the stroke to its own local clients and also publish it to Redis for the others. That is wrong, and the way it fails is instructive: the publishing instance is also subscribed, so it receives its own message and renders the stroke twice. On an opaque pen the duplicate is invisible; with a semi-transparent stroke or an eraser it is not, and it is easily misdiagnosed as a rendering fault.

There is therefore exactly one delivery path: handlers never write to their own sockets, and every instance including the publisher delivers from its subscription. Where a message must skip one connection — the author, who already rendered optimistically — the connection identifier travels inside the published message rather than being filtered locally, so the exclusion works regardless of which instance holds that socket; the author instead receives an acknowledgement carrying the document id and assigned sequence. Undo and redo carry no exclusion, because a visibility change must be re-evaluated by every client including the one that asked for it.

```
async def publish(self, room_code: str, event: dict, skip_conn: str | None = None) -> None:
    """Broadcast an event to every instance serving this room.

    `skip_conn` travels inside the message rather than being applied locally,
    so the exclusion works no matter which instance holds that connection.
    """
    event["_skip"] = skip_conn
    event["_instance"] = settings.instance_id
    await redis().publish(channel(room_code), json.dumps(event, default=str))
```

3. Database Design and JSON Collections

3.1 Collections

Collection	Holds	Documents	Data	Indexes
rooms	One document per board, with participants embedded	8	0.00 MB	0.11 MB
strokes	The append-only log; one document per stroke	80,019	120.3 MB	9.8 MB
snapshots	Rasterised canvas state at a known sequence number	5	13.8 MB	0.04 MB

Review 1 proposed two collections. The third, snapshots, was added after measuring join cost (section 7.2). The average stroke document measures 1,576 bytes, the figure the capacity estimate in section 7.3 is built on.

3.2 The rooms collection

A room carries its settings, a denormalised stroke count for cheap listing, and its participants as an embedded array. Participants are embedded because they are few, bounded by the number of people who can usefully share a board, and always read together with the room itself; a separate collection would add a join to serve no purpose.

```
{
  "id": "6ab1462b336665eb541e4582",
  "code": "LNCSJ5",
  "name": "Seeded board 1",
  "background": "#ffffff",
  "settings": {"gridVisible": true, "width": 1920, "height": 1080},
  "participants": [
    {
      "userId": "seed_user_1",
      "userName": "Devesh",
      "color": "#d97706",
      "lastSeenAt": "2026-09-21T14:58:51.625000+00:00"
```

```
    },
    {
      "userId": "seed_user_2",
      "userName": "Prakrit",
      "color": "#111827",
      "lastSeenAt": "2026-09-21T14:58:51.636000+00:00"
    },
    {
      "userId": "seed_user_3",
      "userName": "Lakshya",
      "color": "#d97706",
      "lastSeenAt": "2026-09-21T14:58:51.646000+00:00"
    },
    {
      "userId": "seed_user_4",
      "userName": "Aarav",
      "color": "#0891b2",
  ... truncated ...
```


3.3 The strokes collection

This is where the flexible-schema argument is made concretely rather than asserted. The three documents below are from the same collection, written by the same code path, with no migration and no nullable columns standing in for absent fields. A pen stroke carries a points array; a rectangle carries a geometry subdocument and no points at all; a text label carries text and font. Fields that do not apply are absent.

A freehand pen stroke


```
{
  "id": "6ab1462b336665eb541e4583",
  "roomId": "6ab1462b336665eb541e4582",
  "seq": 1,
  "userId": "seed_user_1",
  "userName": "Devesh",
  "tool": "pen",
  "style": {"color": "#059669", "width": 6.4, "opacity": 1.0, "blend": "source-over"},
  "active": true,
  "undoneAt": null,
  "undoneBy": null,
  "clientMsgId": null,
  "createdAt": "2026-09-21T11:28:32.932000+00:00",
  "updatedAt": "2026-09-21T11:28:32.932000+00:00",
  "points": [
    {"x": 323.66, "y": 294.22, "p": 0.67},
    {"x": 325.92, "y": 293.95, "p": 0.43},
    {"x": 326.47, "y": 294.99, "p": 0.55},
    "... 145 more points ..."
  ],
  "pointCount": 148,
  ... truncated ...
```


A rectangle, and a text label


```
{
  "rectangle": {
    "id": "6ab1462b336665eb541e4590",
    "roomId": "6ab1462b336665eb541e4582",
    "seq": 14,
    "userId": "seed_user_1",
    "userName": "Devesh",
    "tool": "rect",
    "style": {"color": "#dc2626", "width": 8.3, "opacity": 1.0, "blend": "source-over"},
    "active": true,
    "undoneAt": null,
    "undoneBy": null,
    "clientMsgId": null,
    "createdAt": "2026-09-21T11:28:34.687000+00:00",
    "updatedAt": "2026-09-21T11:28:34.687000+00:00",
    "geometry": {"x": 798.1, "y": 159.31, "w": 417.46, "h": -191.07},
    "bbox": {"minX": 798.1, "minY": -31.76, "maxX": 1215.56, "maxY": 159.31}
  },
  "text": {
    "id": "6ab1462b336665eb541e458a",
    "roomId": "6ab1462b336665eb541e4582",
```

```
    "seq": 8,
    "userId": "seed_user_3",
    "userName": "Lakshya",
    "tool": "text",
    "style": {"color": "#059669", "width": 10.5, "opacity": 1.0, "blend": "source-over"},
    "active": true,
    "undoneAt": null,
    "undoneBy": null,
    "clientMsgId": null,
  ... truncated ...
```


Several fields exist to serve specific queries. The seq field is the authoritative order and the leading component of the replay index. The active flag carries undo state: false means undone but retained. The bbox subdocument is computed on write so that the bounding box of a whole canvas can be answered by an aggregation over small subdocuments without ever reading a point array, and pointCount is precomputed so complexity histograms do not need $size over large arrays.

The eraser deserves a note because the obvious implementation is a trap. Painting the background colour over a stroke looks correct until the board's background changes, at which point every historical erasure becomes a visible white smear. An eraser is therefore stored as an ordinary stroke whose style carries a destination-out blend mode, so it cuts a hole at replay time and remains correct on any background.

3.4 The snapshots collection

A snapshot is a PNG of the canvas as it stood at a known sequence number, stored inline as a data URL together with the sequence it covers, so joining a room costs one snapshot read plus the strokes newer than that sequence. The measured average is 2.8 MB for the seeded boards, inside MongoDB's 16 MB document limit but not with a large margin. That is a worst case: the seeded data is pseudo-random scribble covering the whole canvas and is close to incompressible, whereas a real board is mostly white. If snapshots approached the limit the correct move is GridFS, which chunks large binary values across documents; a cheaper first step would be WebP instead of PNG. Both are noted as future work rather than implemented, because the limit is not currently reached.


```
{
  "id": "6ab14cb13ab4d43d8179f970",
  "roomId": "6ab1462b336665eb541e4582",
  "upToSeq": 48000,
  "strokeCount": 42672,
  "bytes": 2913246,
  "createdAt": "2026-09-21T15:26:41.947000+00:00",
  "image": "<data:image/png;base64 ... 2,913,246 characters ...>"
}
```


3.5 Embedding against referencing

The same database holds one clear example of each strategy, which makes the trade-off easy to state.

Relationship	Strategy	Why
Room to participants	Embedded array	Bounded, small, always read with the room and updated as a unit. Embedding removes a query and keeps the roster atomic with the room document.
Room to strokes	Referenced by roomId	Unbounded. The largest seeded room holds 42,672 strokes at about 1,576 bytes each, far beyond the 16 MB limit. Referencing also lets one stroke be updated or undone without rewriting the room.
Room to snapshots	Referenced by roomId	Large binary payloads with their own lifecycle, pruned independently of the room.

The governing question is not which is faster in the abstract but whether the child collection is bounded. Participants are; strokes are not, and no amount of tuning makes an unbounded embedded array safe.

3.6 The Redis key space

Redis is a database in this system too, and the key-value model was declared in Review 1, so its keys are documented with the same care as the collections.

Key pattern	Type	Purpose	Lifetime
room:{roomId}:seq	String counter	Atomic per-room sequence issued by INCR before each stroke is written.	Persistent; realigned from MongoDB on join.
wb:room:{CODE}	Pub/Sub channel	Carries strokes, previews, cursors, undo, redo, and room updates between instances.	Transient, not stored.
room:{CODE}:presence	Hash	Field per user holding display name, colour, and last cursor position.	Expires after two minutes of silence.

3.7 Validation and schema evolution

Flexible schema does not mean unchecked input. Validation happens at the application boundary through Pydantic models, so a malformed stroke is rejected with a 422 before it reaches the database, while the database itself stays free of a rigid shape. The constraint lives where it can produce a useful error message, and adding a new tool needs no migration: a polygon tool means a model and a renderer, existing documents are untouched, and replay continues to work because readers only look at the fields relevant to each tool.

4. CRUD Implementation

Every operation below is reachable over HTTP as well as over the WebSocket, which is what makes it demonstrable in Postman rather than merely implied by the canvas. A stroke created through the REST endpoint is also published to Redis, so it appears on any open board immediately — a better demonstration than a page refresh.

4.1 Operation map

Verb	Endpoint	Operation
POST	/api/rooms	Create a room with a unique short code
POST	/api/rooms/{code}/strokes	Insert one stroke document
GET	/api/rooms	List boards, most recently active first
GET	/api/rooms/{code}	Read one room with live counts
GET	/api/rooms/{code}/strokes	Replay history, optionally only after a sequence
GET	/api/strokes/{id}	Read a single stroke
PATCH	/api/rooms/{code}	Update name, background, or grid setting
PATCH	/api/strokes/{id}	Update the style of an existing stroke in place
POST	/api/strokes/{id}/move	Translate a shape by mutating nested geometry
POST	/api/rooms/{code}/undo	Soft delete: set active to false, scoped to one user
POST	/api/rooms/{code}/redo	Reactivate the most recently undone stroke
POST	/api/rooms/{code}/compact	Hard delete every stroke left undone
DELETE	/api/strokes/{id}	Hard delete one stroke
DELETE	/api/rooms/{code}/strokes	Hard delete every stroke in a room
DELETE	/api/rooms/{code}	Delete a room and cascade to its strokes and snapshots

4.2 Create

Room creation allocates a six-character code from an alphabet with the ambiguous glyphs removed, so a code can be read aloud without confusion, and retries on the unique index if a collision occurs. Stroke creation is where the flexible schema is exercised: only the fields a given tool uses are written, so a rectangle document has no points key rather than a null one.

```
async def create_stroke(room_id, user_id: str, user_name: str, payload: StrokeCreate) -> dict:
    points = [p.model_dump() for p in payload.points] if payload.points else None

    doc: dict = {
        "roomId": room_id,
        "seq": await next_seq(room_id),
        "userId": user_id,
        "userName": user_name,
        "tool": payload.tool,
        "style": payload.style.model_dump(),
        "active": True,
        "undoneAt": None,
        "undoneBy": None,
        "clientMsgId": payload.clientMsgId,
        "createdAt": now(),
```

4.3 Read

Two reads matter: the full replay used when a room has no snapshot, and one that takes an afterSeq argument and returns only the tail, which is what the snapshot path uses and what makes joining a busy room cheap. Both are served by the same compound index on roomId and seq, because a range query on the second component is exactly what it was built for. Undone strokes are excluded by default and can be requested explicitly, which is how a soft delete is shown to retain data.

```
async def list_strokes(
    room_id, after_seq: int = 0, include_inactive: bool = False, limit: int = 200_000
) -> list[dict]:
    query: dict = {"roomId": room_id, "seq": {"$gt": after_seq}}
    if not include_inactive:
        query["active"] = True
    cursor = db()[STROKES].find(query).sort("seq", 1).limit(limit)
    return [doc async for doc in cursor]


async def list_strokes_capped(room_id, after_seq: int, cap: int) -> tuple[list[dict], bool, int]:
    """Replay strokes after `after_seq`, but never more than `cap` of them.

    A room seeded with 48,000 strokes serialises to roughly 70 MB of JSON, which
```

4.4 Update

Updates operate in place on documents that already exist, the part an append-only log does not give for free. Restyling sets nested fields under style using dot notation, so the point array is not rewritten. Moving a shape uses $inc on nested geometry, so the translation is applied by the server atomically rather than read, modified and written back by the client.

```
async def restyle_stroke(stroke_id: str, patch: StrokeStyleUpdate) -> dict | None:
    """A true in-place update: recolour or resize a stroke already on the canvas."""
    changes = {
        f"style.{field}": value
        for field, value in patch.model_dump(exclude_none=True).items()
    }
    if not changes:
        return await get_stroke(stroke_id)
    changes["updatedAt"] = now()
    return await db()[STROKES].find_one_and_update(
        {"_id": ObjectId(stroke_id)},
        {"$set": changes},
```

```
        return_document=ReturnDocument.AFTER,
    )
```

4.5 Three deletion semantics

Having built all three, the project can contrast them, which is a more useful position than having only the soft delete Review 1 proposed.

Semantics	Mechanism	Data retained	Used for
Soft delete	$set active to false, undoneAt recorded	Yes, fully	Undo and redo, where the stroke must be recoverable
Compaction	deleteMany on strokes still inactive	No	Reclaiming space once a board is finished
Cascade delete	deleteMany on strokes and snapshots, then deleteOne on the room	No	Removing a board entirely

The cascade is where a document store pushes work up into the application. MongoDB has no foreign keys and no ON DELETE CASCADE, so deleting a room without its strokes would silently orphan tens of thousands of documents no query would return and no user could reach. Referential integrity is the application's responsibility, and the endpoint reports how many documents in each collection it removed.

```
async def delete_room(code: str) -> dict:
    """Hard delete with cascade.

    MongoDB has no foreign keys, so the cascade is the application's job. This is
    the clearest place in the project where a document store pushes referential
    integrity up into code, and it is worth saying so during the review.
    """
    room = await get_room_by_code(code)
    if not room:
        return {"deleted": False}

    strokes = await db()[STROKES].delete_many({"roomId": room["_id"]})
    snapshots = await db()[SNAPSHOTS].delete_many({"roomId": room["_id"]})
    await db()[ROOMS].delete_one({"_id": room["_id"]})
```

4.6 Undo as a concurrency problem

Undo looks like the simplest operation in the system and is in fact the only place where a real race exists. Two mistakes are easy to make. The first is scoping undo to the room rather than to the user: a room-wide "undo the last stroke" lets one participant retract another's work, a correctness problem dressed as a feature. Undo is therefore filtered by userId, so each person has their own history.

The second is the read-then-write race. If the handler finds the newest active stroke and then updates it in a second call, two simultaneous undos from the same user can both select the same document, and one of them retracts a stroke the user never asked about. The fix is to make finding and claiming a single atomic operation: findOneAndUpdate with a sort, which matches, updates, and returns in one round trip. The second request no longer matches active: true on that document and moves on to the next one. This is also the query the dedicated undo index exists for — the descending sequence component lets the server stop at the first matching key instead of sorting the room, and section 5 measures the difference.

```
async def undo_last(room_id, user_id: str) -> dict | None:
    """Claim this user's newest active stroke and flag it inactive.

    Scoping by userId matters for correctness, not politeness: a room-wide "undo
    the last stroke" lets one user erase another's work. The single atomic
    findOneAndUpdate also means two simultaneous undos cannot claim the same
    document, because the second one no longer matches `active: True`.
```

```
    """
    return await db()[STROKES].find_one_and_update(
        {"roomId": room_id, "userId": user_id, "active": True},
        {"$set": {"active": False, "undoneAt": now(), "undoneBy": user_id}},
        sort=[("seq", DESCENDING)],
        return_document=ReturnDocument.AFTER,
    )
```

4.7 Verification

The operations are verified by a script rather than by assertion. It walks the whole surface against a running server, including the concurrent-undo race, and reports pass or fail for each check. The transcript below is from an actual run.

```
server: http://127.0.0.1:8052   instance: wb-V   database: whiteboard

CREATE
  [ok]   POST /api/rooms
        room code: JA3JG3
  [ok]   POST stroke (pen) - seq=1
  [ok]   POST stroke (rect) - seq=2
  [ok]   POST stroke (text) - seq=3
  [ok]   POST stroke (eraser) - seq=4
  [ok]   one collection holds several document shapes - 3 distinct field sets

READ
  [ok]   GET /api/rooms/{code} - totalStrokes=4
  [ok]   GET strokes returns them in sequence order - seqs=[1, 2, 3, 4]
  [ok]   GET /api/strokes/{id}
  [ok]   GET strokes?afterSeq= returns only the tail - 2 of 4

UPDATE
  [ok]   PATCH room name and background
  [ok]   PATCH stroke style in place
  [ok]   POST stroke move mutates nested geometry - geometry={'x': 125.0, 'y': 85.0, 'w': 220.0, 'h': 140.0}

SOFT DELETE (undo / redo)
  [ok]   undo flips active to false - seq=4
  [ok]   undone stroke is hidden from the default read
  [ok]   undone stroke is still on disk
  [ok]   undo is scoped per user, so it cannot touch another user's work - nothing left to undo for this user
  [ok]   redo reactivates it
  [ok]   concurrent undos claim different documents - claimed ['6ab1550e64d773ccafe4f5c9', '6ab1550e64d773ccafe4f5c8']

AGGREGATION and INDEXES
  [ok]   aggregation returns rows with its pipeline - 1 rows in 2.88 ms
  [ok]   $facet returns several result sets at once - facets=['byTool', 'recent', 'topContributors', 'totals']
  [ok]   $lookup joins rooms to strokes
  [ok]   explain reports an index scan - IXSCAN on ix_stroke_replay, docsExamined=4
  [ok]   every planned stroke index exists - 5 indexes

HARD DELETE
  [ok]   compaction turns soft deletes into real deletions - purged 2, 4 -> 2
  [ok]   DELETE /api/strokes/{id}
  [ok]   deleted stroke is really gone
  [ok]   DELETE room cascades to its strokes and snapshots - strokes=1, snapshots=0
  [ok]   room is gone
  [ok]   its strokes are gone too

all checks passed
```

5. Indexing and Query Plans

5.1 The indexes and why each exists

Every index below was added to serve a specific query in the application, and none was added speculatively. Indexes are not free: they consume storage, and they slow every write because each insert must update each one.

Index	Collection	Key	Why it exists
ix_room_code	rooms	{code: 1}	Every join is a lookup by short room code; unique prevents duplicate codes.
ix_room_activity	rooms	{lastActivityAt: -1}	Powers the 'recent rooms' list without scanning the whole collection.

ix_stroke_replay	strokes	{roomId: 1, seq: 1}	The hot path: replay a room's strokes in order. Covers range queries on seq for incremental loading after a snapshot.
ix_stroke_undo	strokes	{roomId: 1, userId: 1, active: 1, seq: -1}	Per-user undo claims the newest active stroke belonging to that user. Descending seq lets the atomic findOneAndUpdate stop at the first hit.
ix_stroke_tool	strokes	{roomId: 1, tool: 1}	Supports the tool-distribution aggregation without a collection scan.
ix_stroke_time	strokes	{roomId: 1, createdAt: 1}	Supports the time-bucketed activity aggregation ($dateTrunc grouping).
ix_snapshot_latest	snapshots	{roomId: 1, upToSeq: -1}	Fetches the newest snapshot for a room in a single index seek.

5.2 Method

Claiming an index helps is not the same as showing it. The application exposes a benchmark endpoint that drops the non-unique indexes, runs each query five times and keeps the best time, captures the query plan with explain in executionStats mode, then rebuilds the indexes and repeats. Every query benchmarked is one the application genuinely issues, with the same sort, projection and limit.

An earlier version of this benchmark was misleading and the correction is worth recording. It fetched five thousand full documents for every case, and because a stroke document averages 1,576 bytes, the cost of transferring and decoding point arrays swamped the cost of finding them, so the index appeared to deliver almost nothing. Timing the real queries instead — a sorted lookup limited to one document, a bounded range scan, a projection, a count — shows where the index decides the outcome.

5.3 Results

Query	Plan without	Docs exam.	Plan with	Docs exam.	Index used	Without	With	Speed-up
undo lookup	COLLSCAN	80,019	IXSCAN	1	ix_stroke_undo	59.4 ms	2.6 ms	23.2x
incremental join	COLLSCAN	80,019	IXSCAN	500	ix_stroke_replay	81.1 ms	22.1 ms	3.7x
history metadata	COLLSCAN	80,019	IXSCAN	48,000	ix_stroke_replay	305.1 ms	274.5 ms	1.1x
tool count	COLLSCAN	80,019	COUNT_SCAN	0	ix_stroke_tool	52.7 ms	11.0 ms	4.8x
covered query	COLLSCAN	80,019	IXSCAN	0	ix_stroke_tool	156.9 ms	94.5 ms	1.7x

Table 5.2 — Every benchmarked query with the indexes dropped and rebuilt. Plans and documents examined come from explain; timings are the best of five runs.

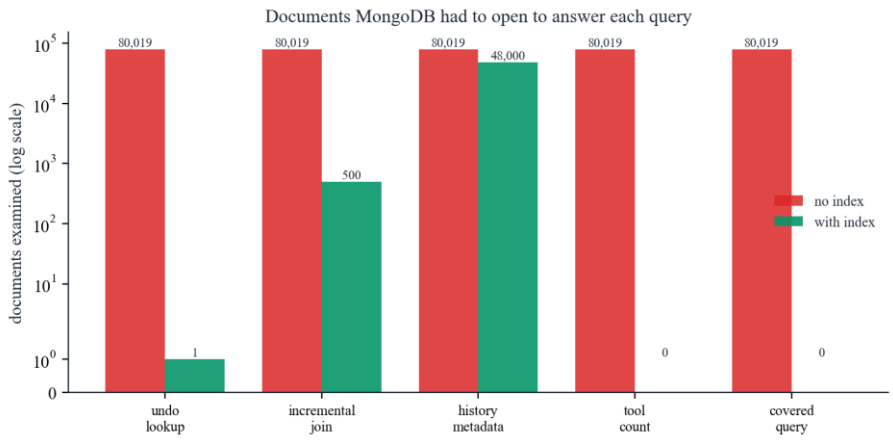


Figure 5.1 — Documents MongoDB had to open, on a logarithmic scale. This is planner output rather than a timing, so it is reproducible.

The documents-examined column is the primary evidence because it is deterministic. The undo lookup goes from opening every document in the collection to opening exactly one, and the incremental join from the whole collection to the five hundred documents it wants. Two queries reach zero documents examined, and the plan column says why: the count wins a COUNT_SCAN, which tallies index keys without fetching anything, and the covered query requests only fields already held in the index, so MongoDB never touches the collection at all.

The timings are secondary because they are noisy in a way the reader should know about. The whole strokes collection is 120 MB and fits comfortably in the WiredTiger cache on this machine, so an unindexed scan is limited by memory bandwidth rather than disk, which flatters the unindexed case. On a memory-constrained tier such as Atlas M0 the gap would be wider, so the reported speed-ups are a conservative floor.

5.4 Where the index barely helps

One result is close to negative and is reported rather than omitted. The history metadata query was 305 ms without the index and 274 ms with it, a speed-up of 1.1x — effectively no gain, and the documents-examined column confirms why: it still opens 48,000 documents. This query is not selective, and walking an index and following each key to its document is random access where scanning the collection in storage order is sequential. When a query touches most of a collection the sequential scan wins, and MongoDB's own planner would often choose it. Indexes pay off in proportion to how much of the collection they let the server skip, and a query that skips nothing gains nothing.

5.5 What the indexes cost

Index	Size on disk
_id_	7,140 KB
ix_stroke_time	812 KB
ix_stroke_undo	796 KB
ix_stroke_replay	740 KB
ix_stroke_tool	536 KB

The four purpose-built indexes plus the default _id index total 9.8 MB against 120 MB of stroke data, about 8 per cent overhead, which is a reasonable price. The _id index is the largest single index and is never used by any query here — every read is by roomId, by sequence, or by user. MongoDB requires it, so it cannot be dropped, but it is a reminder that the default is not free.

6. Aggregation Queries

Nine pipelines are implemented. Each answers a question the application genuinely asks, and each is reachable from the analytics panel in the running interface, which displays the pipeline next to its own output — the API returns the pipeline with every result for exactly that reason.

Pipeline	Question it answers	Stages	Rows	Time
Contribution breakdown by user	Who drew how much in this room, and how often did they undo?	$match, $group, $project, $sort	5	351 ms
Tool usage mix	Which tools dominate, and how complex is a typical stroke for each?	$match, $group, $project, $sort	6	166 ms
Drawing activity over	How did drawing intensity vary minute by	$match, $group, $project, $sort,	109	159 ms

time	minute?	$limit		
Stroke complexity distribution	Are strokes mostly short taps or long freehand curves?	$match, $bucket, $project	6	238 ms
Colour palette in use	Which colours carry the most ink on the active canvas?	$match, $group, $project, $sort, $limit	8	188 ms
Content bounding box	What canvas region actually contains drawing?	$match, $group, $project	1	191 ms
Room leaderboard (joined)	Across all rooms, which are the busiest?	$match, $lookup, $set, $project, $sort, $limit	8	354 ms
Single-request dashboard	Can the whole analytics panel load in one query?	$match, $facet	1	420 ms
Storage footprint by tool	How many bytes does each kind of stroke document cost?	$match, $set, $group, $project, $sort	6	219 ms

Table 6.1 — Every pipeline, measured against the profiled room.

6.1 Contribution breakdown by user

Who drew how much in this room, and how often did they undo?

The pipeline the analytics panel opens on. The detail worth noticing is $cond inside $group: active and undone strokes are counted in a single pass over the matched documents rather than by issuing two queries and subtracting. The undo rate falls out of the same pass, and $addToSet collects each person's tool set without a second lookup.

```
[
  {"$match": {"roomId": "6ab1462b336665eb541e4582"}},
  {
    "$group": {
      "id": {"userId": "$userId", "userName": "$userName"},
      "totalStrokes": {"$sum": 1},
      "activeStrokes": {"$sum": {"$cond": ["$active", 1, 0]}},
      "undoneStrokes": {"$sum": {"$cond": ["$active", 0, 1]}},
      "totalPoints": {"$sum": {"$ifNull": ["$pointCount", 0]}},
      "toolsUsed": {"$addToSet": "$tool"},
      "firstStrokeAt": {"$min": "$createdAt"},
      "lastStrokeAt": {"$max": "$createdAt"}
    }
  },
  {
    "$project": {
      "id": 0,
      "userId": "$_id.userId",
      "userName": "$_id.userName",
      "totalStrokes": 1,
      "activeStrokes": 1,
      "undoneStrokes": 1,
      "totalPoints": 1,
      "toolsUsed": 1,
      "avgPointsPerStroke": {"$round": [{"$divide": ["$totalPoints", {"$max": ["$totalStrokes", 1]}]}, 1]},
      "undoRatePct": {
        "$round": [{"$multiply": [{"$divide": ["$undoneStrokes", {"$max": ["$totalStrokes", 1]}]}, 100]}, 1]
      },
      "activeSeconds": {"$round": [{"$divide": [{"$subtract": ["$lastStrokeAt", "$firstStrokeAt"]}, 1000]}, 0]}
    }
  },
  {"$sort": {"totalStrokes": -1}}
]
```

Result — 5 rows in 351 ms:

User	Strokes	Active	Undone	Points	Avg pts	Undo %
Devesh	19,263	17,086	2,177	545,286	28.3	11.3
Prakrit	12,807	11,393	1,414	366,520	28.6	11.0
Lakshya	9,502	8,474	1,028	268,634	28.3	10.8
Meera	3,269	2,920	349	91,284	27.9	10.7

6.2 Stroke complexity distribution

Are strokes mostly short taps or long freehand curves?

$bucket produces a histogram server-side; the equivalent in SQL needs a CASE expression per band or a join against a bands table. The result confirms the long-tailed distribution the generator was designed to produce: most strokes are short, a few are very long, and the tail is what drives document size.

```
[
  {"$match": {"roomId": "6ab1462b336665eb541e4582", "pointCount": {"$exists": true}}},
  {
    "$bucket": {
      "groupBy": "$pointCount",
      "boundaries": [0, 10, 25, 50, 100, 250, 1000],
      "default": "1000+",
      "output": {
        "strokes": {"$sum": 1},
        "totalPoints": {"$sum": "$pointCount"},
        "avgWidth": {"$avg": "$style.width"}
      }
    }
  },
  {
    "$project": {
      "id": 0,
      "minPoints": "$_id",
      "strokes": 1,
      "totalPoints": 1,
      "avgWidth": {"$round": ["$avgWidth", 2]}
    }
  }
]
```

Result — 6 rows in 238 ms:

Lower bound (points)	Strokes	Total points	Avg width
0	9,831	53,835	8.6
10	13,417	200,962	8.8
25	7,126	263,300	8.7
50	4,416	274,716	8.6

6.3 Room leaderboard (joined)

Across all rooms, which are the busiest?

The only pipeline that starts from rooms rather than strokes. $lookup joins the two collections, and the sub-pipeline inside the join aggregates the matched strokes before they are returned, so tens of thousands of stroke documents are never materialised into the parent room document. Without the sub-pipeline this query would try to build an array far larger than the 16 MB document limit and fail.

```
[
  {"$match": {"isArchived": false}},
  {
    "$lookup": {
      "from": "strokes",
      "localField": "_id",
      "foreignField": "roomId",
      "as": "strokeStats",
      "pipeline": [
        {
          "$group": {
            "id": null,
            "total": {"$sum": 1},
            "active": {"$sum": {"$cond": ["$active", 1, 0]}},
            "points": {"$sum": {"$ifNull": ["$pointCount", 0]}},
            "contributors": {"$addToSet": "$userId"}
          }
        }
      ]
    }
  },
  {"$set": {"stats": {"$first": "$strokeStats"}}},
  {
    "$project": {
      "id": 0,
```

```
      "code": 1,
      "name": 1,
      "createdAt": 1,
      "participantCount": {"$size": "$participants"},
      "totalStrokes": {"$ifNull": ["$stats.total", 0]},
      "activeStrokes": {"$ifNull": ["$stats.active", 0]},
      "totalPoints": {"$ifNull": ["$stats.points", 0]},
      "contributorCount": {"$size": {"$ifNull": ["$stats.contributors", []]}}
    }
  },
  {"$sort": {"totalStrokes": -1}},
  {"$limit": 20}
]
```

Result — 8 rows in 354 ms:

Room	Name	People	Strokes	Active	Contributors
LNCSJ5	Seeded board 1	5	48,000	42,672	5
S36UAR	Seeded board 2	5	8,000	7,182	5
K9KE5J	Seeded board 3	5	8,000	7,125	5
TEJWCQ	Seeded board 4	5	8,000	7,134	5

6.4 Single-request dashboard

Can the whole analytics panel load in one query?

```
[
  {"$match": {"roomId": "6ab1462b336665eb541e4582"}},
  {
    "$facet": {
      "totals": [
        {
          "$group": {
            "id": null,
            "strokes": {"$sum": 1},
            "active": {"$sum": {"$cond": ["$active", 1, 0]}},
            "points": {"$sum": {"$ifNull": ["$pointCount", 0]}},
            "contributors": {"$addToSet": "$userId"},
            "maxSeq": {"$max": "$seq"}
          }
        },
        {
          "$project": {
            "id": 0,
            "strokes": 1,
            "active": 1,
            "points": 1,
            "maxSeq": 1,
            "contributorCount": {"$size": "$contributors"}
          }
        }
      ],
      "byTool": [
        {"$group": {"id": "$tool", "count": {"$sum": 1}}},
        {"$project": {"id": 0, "tool": "$_id", "count": 1}},
        {"$sort": {"count": -1}}
      ],
      "topContributors": [
        {"$group": {"id": "$userName", "strokes": {"$sum": 1}}},
        {"$project": {"id": 0, "userName": "$_id", "strokes": 1}},
        {"$sort": {"strokes": -1}},
        {"$limit": 5}
      ],
      "recent": [
        {"$sort": {"seq": -1}},
        {"$limit": 5},
        {
          "$project": {"id": 0, "seq": 1, "tool": 1, "userName": 1, "active": 1, "color": "$style.color"}
        }
      ]
    }
  }
]
```

This pipeline answers a performance question rather than a data question. The panel needs four summaries at once, and issuing four queries means four round trips and four passes over the same matched documents. $facet evaluates all four sub-pipelines over one $match, so the panel

loads from a single request. The output below is one document whose fields are the four result sets.

facet: totals

strokes	active	points	maxSeq	contributorCount
48,000	42,672	1,362,497	48,000	5

facet: byTool

count	tool
32,598	pen
4,272	eraser
3,898	rect

The remaining facets in the same response — topContributors, recent — are shown in the interface (Figure 8.4).

6.5 The remaining pipelines

The five below are implemented and demonstrable in the interface on the same terms as those above. They are summarised rather than expanded because each is a variation on stages already discussed; their stage lists, row counts and timings are in Table 6.1 and their output is shown in the interface (Figure 8.4).

Tool usage mix

Which tools dominate, and how complex is a typical stroke for each? *$match → $group → $project → $sort*

A straightforward $group, but the pipeline that makes the flexible-schema argument measurable: it aggregates across documents with genuinely different fields and reports average point counts per tool, which differ by an order of magnitude between a freehand stroke and a rectangle.

Drawing activity over time

How did drawing intensity vary minute by minute? *$match → $group → $project → $sort → $limit*

$dateTrunc buckets timestamps into one-minute bins inside the database, so the client receives a few hundred aggregated rows instead of tens of thousands of raw timestamps. Doing this in the application would mean transferring the entire collection to draw a chart.

Colour palette in use

Which colours carry the most ink on the active canvas? *$match → $group → $project → $sort → $limit*

A computed metric inside an accumulator. Ink is not a stored field — it is stroke width multiplied by point count, summed with $sum — so aggregation derives a quantity the schema never recorded.

Content bounding box

What canvas region actually contains drawing? *$match → $group → $project*

$min and $max over the precomputed bbox subdocuments answer which region of the canvas contains drawing without reading a single point array. This is the payoff for computing bbox at write time, and it backs the Fit to content button.

Storage footprint by tool

How many bytes does each kind of stroke document cost? $match → $set → $group → $project → $sort

$bsonSize measures the real on-disk size of each document, which the capacity projection in section 7.3 is built on, and quantifies the cost of storing point arrays inline.

6.6 What these queries would cost in SQL

It is fair to ask whether any of this needs a document database, since PostgreSQL has grouping and JSON columns too. Most of these pipelines could be expressed in SQL; the interesting difference is not expressiveness but the shape of the data they run over. A relational schema for this workload would either store one row per point, turning a single stroke into dozens of rows and the contributors query into an aggregate over millions, or store the point array in a JSON column, at which point the schema is a document store with extra steps and the indexes are harder to reason about. The pipelines here run directly over the documents the application writes and reads, with no mapping layer between the two.

7. Performance and Scalability

7.1 The dataset

Review 1 estimated 50,000 to 100,000 stroke documents. Nobody draws that by hand, and indexing and aggregation results are meaningless on a few hundred documents, so a generator produces a realistically shaped corpus: a long tail of short strokes with a few very long ones, a mix of tools, about one stroke in nine left undone so the undo analytics have real numbers, and timestamps spread over hours for the time-bucketed pipeline to bucket.

Property	Measured
Stroke documents	80,019
Rooms	8
Average document size	1,576 bytes
Data size	120.3 MB
Storage size on disk	162.4 MB
Total index size	9.8 MB
Largest single room	42,672 active strokes

7.2 The cost of joining a room, and why snapshots were built

Replaying an append-only log is elegant and has one obvious weakness: the cost of a join grows without bound as a board is used. Review 1 listed snapshotting as optional; measuring the join settled the question. Joining the largest room means reading 42,672 stroke documents, which is 66.0 MB of JSON. The query itself takes about 4042 ms, but serialising that many documents to JSON takes roughly 9 seconds, and the client then has to parse it all and render every stroke. A board that takes twelve seconds to open is not usable.

Room	Active strokes	Full replay	Query	Tail	Snapshot	Snapshot join	Reduction
LNCSJ5	42,672	66.0 MB	4042 ms	438	2845 KB	3.45 MB	95%

SFBFLQ	7,188	11.1 MB	545 ms	447	2783 KB	3.42 MB	69%
S36UAR	7,182	11.1 MB	574 ms	455	2794 KB	3.46 MB	69%
TEJWCQ	7,134	11.3 MB	705 ms	447	2796 KB	3.46 MB	70%
K9KE5J	7,125	11.3 MB	641 ms	439	2889 KB	3.57 MB	68%

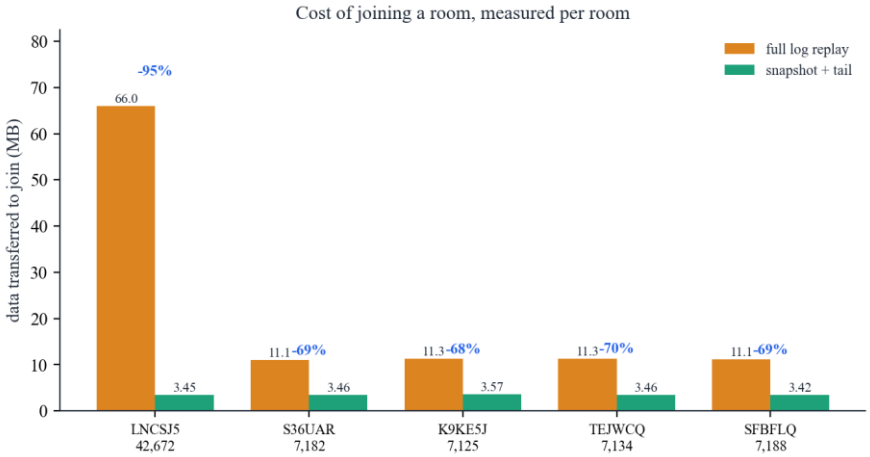


Figure 7.1 — Data transferred to join each room, measured.

A snapshot is a PNG of the canvas at a known sequence number, so joining costs one snapshot plus only the strokes newer than it: for the largest room 3.45 MB instead of 66.0 MB, a reduction of 95 per cent. Measured in the browser, joining that room now completes in roughly half a second.

Two details are worth recording. The snapshot does not replace the log: the strokes remain, so history, undo and analytics still work, and a snapshot can be regenerated or discarded at any time. It is a cache of derived state, which is the role a rasterised canvas should have. Snapshots are also counterproductive below a threshold — for a board of ten strokes the image is far larger than the strokes it replaces, and the table above shows the reduction going sharply negative for small rooms — so the implementation snapshots only every few hundred strokes and the server-side generator skips rooms below a floor.

Snapshots are produced two ways. During normal use the client rasterises its own canvas and uploads it, which costs the server nothing. That is no help for a generated room, since creating the first snapshot would require the very full replay the snapshot exists to avoid, so a server-side renderer using Pillow closes the loop. That path is also the one a production system would prefer, because it does not trust a client to rasterise authoritative state.

7.3 Capacity on the free Atlas tier

Review 1 nominated MongoDB Atlas M0, which allows 512 MB. At the measured 1,576 bytes per document, 100,000 strokes is about 150 MB of data and, at the measured index overhead of 8 per cent, roughly 163 MB in total. That is 32 per cent of the quota, so the stated target fits with room to spare.

Snapshots, not strokes, are the component that threatens the quota. At the measured 2.8 MB per snapshot for worst-case scribble, a handful of busy boards would consume more than the entire stroke log. Pruning to the three most recent snapshots per room is implemented for this reason, and it is the right default: older snapshots are strictly redundant, since any of them plus the log reconstructs the same canvas.

7.4 Write throughput and where the limits are

The separation of previews from commits keeps the write rate proportional to strokes rather than to pointer samples. A person drawing continuously produces a few strokes per second, not the sixty pointer events per second their mouse generates, so a room of ten active participants writes on the order of tens of documents per second — comfortable for a single MongoDB node. The load that scales with pointer events lands on Redis Pub/Sub instead, where messages are small and nothing is persisted.

The known ceiling is the fanout itself. Every preview from every user is delivered to every other participant, so traffic within a room grows with the square of the number of participants. For a whiteboard of a handful of people this is not a practical limit; a system expecting hundreds of simultaneous editors in one room would need to throttle or aggregate previews server-side, which section 9.3 records as a known limitation rather than solved.

8. Prototype Walkthrough

The screenshots below are from the running system against the seeded database. Each is annotated with what it is evidence of.

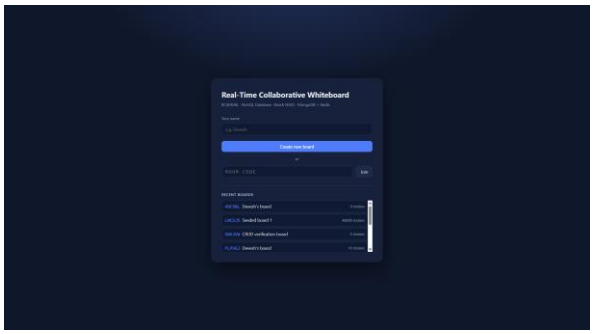


Figure 8.1 — The lobby. Boards are listed by last activity, served by the ix_room_activity index rather than by scanning the collection.

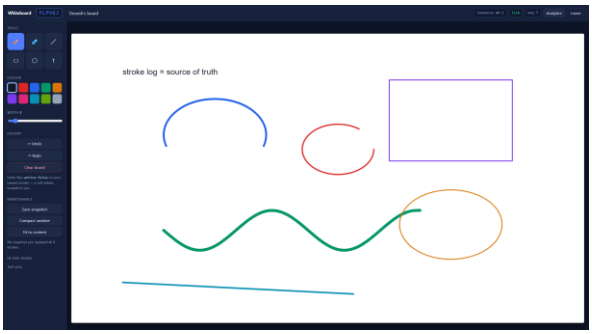


Figure 8.2 — One board with all six tools used: pen, eraser, line, rectangle, ellipse and text. Every mark is a separate document, and the six tools produce documents with different field sets in one collection.

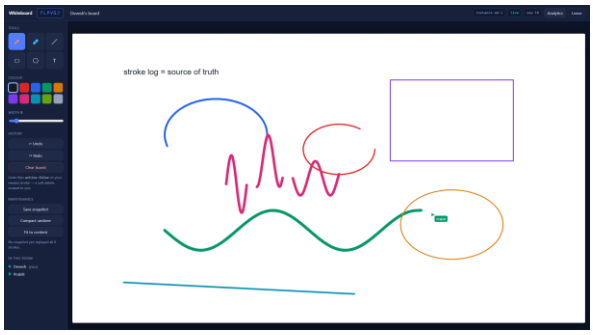


Figure 8.3 — The result that matters most. This window is connected to instance wb-1, shown in the badge at the top right. The pink strokes and the labelled cursor belong to a user whose browser is connected to wb-2, a separate operating-system process sharing no memory with this one. Both reached this screen through Redis Pub/Sub, which is the claim the architecture makes.



Figure 8.4 — The analytics panel, showing the contributors pipeline next to the output it produced and the server-side execution time. The canvas behind it is the 42,672-stroke seeded board restored from a snapshot.



Figure 8.5 — The index benchmark run from the interface. For each query it reports the plan, the documents examined and the time, with the indexes dropped and then rebuilt.

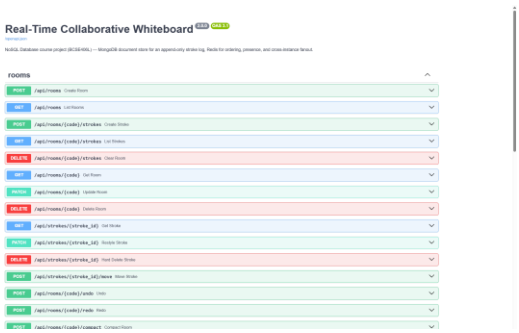


Figure 8.6 — The generated API documentation, showing the full CRUD surface used to exercise the operations in Postman. Live collection and index statistics are also available under the Storage tab.

9. Progress Report

9.1 Completed between Review 1 and Review 2

- Full application built: FastAPI backend, WebSocket transport, and a canvas client with six drawing tools, live cursors, presence, undo and redo.
- Three collections implemented, with the stroke log holding several document shapes and no schema migration between them.
- Complete CRUD surface over both REST and WebSocket, including operations that genuinely update and genuinely delete, plus a cascading room delete.
- Seven purpose-built indexes, each justified against a specific query, with plans captured through explain.
- Nine aggregation pipelines, surfaced in the application itself rather than only in a shell, each displayed alongside the pipeline that produced it.
- A generated dataset of 80,019 stroke documents, shaped to resemble real drawing rather than uniform noise.
- A measured index benchmark and join-cost study, with snapshotting implemented in response to what the join-cost study showed.
- Cross-instance operation demonstrated with two independent backend processes, and an automated verification script covering every operation including the concurrent-undo race.

9.2 Deviations from the Review 1 proposal

Four things differ from the Review 1 information sheet. Each was a considered decision, and recording them is more useful than quietly matching the original document.

Review 1 stated	Review 2 actual	Reason
Two collections: rooms and strokes	Three collections: rooms, strokes and snapshots	Measurement showed a full replay of the largest room transfers 66 MB. Snapshots cut that by about 95 per cent, so the optional optimisation became part of the normal join path and needed its own collection.
Snapshotting listed as optional	Implemented, with both client-side and server-side generation	Same reason. It is no longer optional because a large board is unusable without it.
Cloud hosting on MongoDB Atlas free tier	Local MongoDB and Redis in Docker for development; Atlas-ready by configuration	Atlas is a one-line change in the environment file and the capacity estimate in section 7.3 targets M0. Local Docker keeps the demonstration independent of network conditions, and Redis has no hosting listed in Review 1 at all, so it needed one either way.
Postman for API testing	Postman for REST, plus a scripted	Postman is awkward for WebSockets and cannot easily

	verification run	express the concurrent-undo race, which needs two requests genuinely in flight at once.

9.3 Known limitations

Stating these is more useful than overclaiming; each is acceptable at this stage.

- **No authentication.** Identity is a display name in browser storage and anyone with a room code can join, so per-user undo trusts the client's claimed identity. Real accounts would be required before deployment but add nothing to the database concepts under assessment.
- **Preview fanout grows with the square of participants in a room.** Acceptable for a whiteboard used by a handful of people; a larger room would need server-side throttling.
- **Snapshots of dense boards approach a size where GridFS would be the right storage.** The 16 MB limit is not reached and pruning keeps the total bounded, but the headroom is measured rather than generous.
- **No conflict resolution for concurrent modification of the same stroke.** Two users restyling one stroke resolve last-write-wins. This is where operational transformation or a CRDT would genuinely be needed, and it is not claimed.
- **Timing measurements are cache-warm,** which as section 5.3 explains understates rather than overstates the benefit of the indexes.

9.4 Plan for Review 3

- Deploy against MongoDB Atlas M0 and a hosted Redis and re-measure every query, since network latency and a constrained cache will change the numbers in section 5 and are likely to widen the index advantage.
- Add a TTL index to expire abandoned rooms automatically — a NoSQL feature the project has a genuine use for and does not yet exercise.
- Replace the polling heartbeat with MongoDB change streams for a second, database-driven notification path, and compare it with the Redis path.
- Move snapshot storage to GridFS and compare WebP against PNG, closing the headroom concern in section 9.3.
- Add shared draggable objects, which would introduce genuine concurrent modification and so make an operational-transformation or CRDT layer necessary rather than ornamental.
- Load-test with scripted WebSocket clients to find where a single instance saturates, and add a third instance to confirm the fanout scales as claimed.

10. Source Code Listings

Selected extracts, quoted directly from the files that run. The complete source is in the project repository; the layout is given first so the extracts can be located.

```
whiteboard/
├─ app/
│   ├─ main.py             FastAPI application, static mount, startup
│   ├─ config.py           environment configuration
│   ├─ db.py               Mongo and Redis clients, index definitions
│   ├─ models.py           Pydantic request and response models
│   ├─ util.py             room codes, BSON serialisation, bounding boxes
│   ├─ aggregations.py     the nine pipelines and their metadata
│   ├─ repositories/       rooms.py, strokes.py, snapshots.py
│   ├─ realtime/           hub.py (Redis fanout), ws.py (socket protocol)
│   └─ routers/            rooms.py, analytics.py, admin.py
```

```
├─ static/                index.html, css/, js/ (app, board, net, analytics)
├─ scripts/               seed, benchmark, snapshot, measurement, report, deck
└─ docker-compose.yml     MongoDB replica set and Redis
```

10.1 Index definitions

Indexes are declared as data rather than as a sequence of createIndex calls, each entry pairing the key specification with the rationale quoted in Table 5.1 so the two cannot drift apart. The remaining entries follow the same shape.

```
INDEX_PLAN: list[tuple[str, list[tuple[str, int]], dict, str]] = [
    (
        ROOMS,
        [("code", ASCENDING)],
        {"name": "ix_room_code", "unique": True},
        "Every join is a lookup by short room code; unique prevents duplicate codes.",
    ),
    (
        ROOMS,
        [("lastActivityAt", DESCENDING)],
        {"name": "ix_room_activity"},
        "Powers the 'recent rooms' list without scanning the whole collection.",
    ),
    (
        STROKES,
        [("roomId", ASCENDING), ("seq", ASCENDING)],
```

10.2 The WebSocket protocol

```
if kind == "commit":
    payload = StrokeCreate(**msg.get("stroke", {}))
    stroke = await strokes_repo.create_stroke(room_id, user_id, user_name, payload)
    await rooms_repo.touch_room(room_id, stroke_delta=1)

    # The author already rendered this stroke optimistically, so it gets a
    # lightweight ack with the authoritative seq while everyone else gets the
    # full document.
    await ws.send_json(
        {
            "type": "ack",
            "clientMsgId": payload.clientMsgId,
            "id": str(stroke["_id"]),
            "seq": stroke["seq"],
        }
    )
    await hub.publish(code, {"type": "stroke", "stroke": jsonable(stroke)}, skip_conn=conn_id)
    return

if kind in ("undo", "redo"):
```

10.3 Incremental canvas rendering

```
/** Repaint the offscreen base from scratch. Only needed on structural change. */
rebuild() {
  this.baseCtx.setTransform(1, 0, 0, 1, 0, 0);
  this.baseCtx.clearRect(0, 0, this.base.width, this.base.height);
  if (this.snapshotImage) {
    this.baseCtx.drawImage(this.snapshotImage, 0, 0, this.base.width, this.base.height);
  }
  for (const stroke of this.strokes) this._path(this.baseCtx, stroke);
  this.invalidate();
}

appendStroke(stroke) {
```

The aggregation pipelines appear in full in section 6 and the Redis fanout loop in section 2.7.

11. References

The Review 1 list included four items that are not peer-reviewed papers: one with authors withheld, an encyclopedia entry and two blog posts. Those are replaced below. The operational transformation literature is retained because section 2.6 argues about its applicability rather than citing it in passing.

[1] A. K. Singh, P. K. Singh, M. Kumar, S. Tiwari, and V. Jain, "A comparative study of MongoDB and document-based MySQL for big data application data management," Big Data and Cognitive Computing, vol. 6, no. 2, p. 49, 2022.

[2] C. Győrödi, R. Győrödi, G. Pecherle, and A. Olah, "A comparative study: MongoDB vs. MySQL," in Proc. 13th Int. Conf. on Engineering of Modern Electric Systems (EMES), 2015, pp. 1-6.

[3] A. Gupta, S. Tyagi, N. Panwar, S. Sachdeva, and U. Saxena, "NoSQL databases: Critical analysis and comparison," in Proc. Int. Conf. on Computational and Communication Technologies (IC3TSN), 2017, pp. 293-299.

[4] Z. Wei-ping, L. Ming-xin, and C. Huan, "Using MongoDB to implement textbook management system instead of MySQL," in Proc. IEEE Int. Conf. on Communication Software and Networks, 2011, pp. 303-305.

[5] Y. Li and S. Manoharan, "A performance comparison of SQL and NoSQL databases," in Proc. IEEE Pacific Rim Conf. on Communications, Computers and Signal Processing (PACRIM), 2013, pp. 15-19.

[6] D. Sun and C. Sun, "Context-based operational transformation in distributed collaborative editing systems," IEEE Transactions on Parallel and Distributed Systems, vol. 20, no. 10, pp. 1454-1470, 2009.

[7] C. Sun and C. Ellis, "Operational transformation in real-time group editors: issues, algorithms, and achievements," in Proc. ACM Conf. on Computer Supported Cooperative Work (CSCW), 1998, pp. 59-68.

[8] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-free replicated data types," in Proc. 13th Int. Symp. on Stabilization, Safety, and Security of Distributed Systems (SSS), 2011, pp. 386-400.

[9] M. Weiss, P. Urso, and P. Molli, "Logoot: A scalable optimistic replication algorithm for collaborative editing on P2P networks," in Proc. IEEE Int. Conf. on Distributed Computing Systems (ICDCS), 2009, pp. 404-412.

[10] N. Preguiça, J. M. Marquès, M. Shapiro, and M. Letia, "A commutative replicated data type for cooperative editing," in Proc. IEEE Int. Conf. on Distributed Computing Systems (ICDCS), 2009, pp. 395-403.

[11] S. Gilbert and N. Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services," ACM SIGACT News, vol. 33, no. 2, pp. 51-59, 2002.

[12] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in Proc. NetDB Workshop, 2011, pp. 1-7.

Software documentation consulted during implementation — the MongoDB Manual (aggregation pipeline, indexing strategies, explain results), the Redis documentation (Pub/Sub, INCR, key expiry) and the FastAPI documentation (WebSockets) — is reference material rather than part of the literature survey.